Take-Home Assignment

Data Extraction & Structuring

Enron Email Dataset — AI-Assisted Pipeline with MCP Integration

Using Claude Code or any AI coding tool of your choice

Deadline: 7 days from receipt

1. Overview

This assignment evaluates your ability to use AI-assisted development tools (preferably Claude Code) to build a data extraction, structuring, and automated notification pipeline. You will work with the Enron Email Dataset — one of the most widely used real-world unstructured datasets — to parse raw email files, extract structured fields, store them in a local database, detect duplicates, and send automated notification emails using MCP (Model Context Protocol) server integration.

Goal: Demonstrate that you can leverage AI coding tools effectively to solve a real data engineering problem end-to-end — from raw data ingestion to automated action. We want to see how you prompt, iterate, debug, and refine with AI assistance, and how you integrate external services via MCP.

2. Dataset

Source: Enron Email Dataset

Download: https://www.cs.cmu.edu/~enron/enron_mail_20150507.tar.gz

Size: Approximately 500,000+ email files across 150 employee mailboxes

Format: Raw RFC 2822 email text files organized in a nested folder structure: maildir/<employee_name>/<folder>/<email_file>

Important: You do NOT need to process the entire dataset. A representative subset of at least 5 employee mailboxes (minimum 10,000 emails total) is sufficient to demonstrate your pipeline. Document which mailboxes you selected and why.

3. Field Definitions

Each email must be parsed to extract the following fields. Note that the raw data is inconsistent — not every email contains all fields, and formatting varies significantly.

3.1 Mandatory Fields (Must Extract)

These fields must be present for every record stored in the database. If a mandatory field cannot be extracted, log the email as a failed parse with the reason.

Field Name	Data Type	Description
message_id	STRING (unique)	The Message-ID header. Must be unique in DB.
date	DATETIME	Parsed and normalized to UTC. Handle timezone abbreviations (PST, EST, CDT, etc.) and offset formats.
from_address	STRING	Sender email address. Extract just the address, not the display name.
to_addresses	LIST of STRING	All recipients in the To field. Handle comma-separated lists, line continuations, and mixed formats.
subject	STRING	Full subject line. Preserve Re:/Fwd: prefixes.
body	TEXT	The email body content. Separate from forwarded/quoted content where possible.
source_file	STRING	Relative path to original file (e.g., maildir/lay-k/inbox/45) for traceability.

3.2 Optional Fields (Extract If Present)

These fields should be extracted when they exist. Store NULL/empty if not present.

Field Name	Data Type	Description
cc_addresses	LIST of STRING	CC recipients. Same parsing rules as to_addresses.
bcc_addresses	LIST of STRING	BCC recipients. Rarely present but must be captured.
x_from	STRING	Display name from the X-From header (Enron-specific).
x_to	STRING	Display name(s) from the X-To header.
x_cc	STRING	Display name(s) from the X-cc header.
x_bcc	STRING	Display name(s) from the X-bcc header.
x_folder	STRING	The X-Folder header value indicating mailbox folder.
x_origin	STRING	The X-Origin header value.
content_type	STRING	MIME content type if specified.
has_attachment	BOOLEAN	Inferred from Content-Type, MIME boundaries, or body references to attachments.
forwarded_content	TEXT	Extracted forwarded email content, separated from the primary body.

quoted_content	TEXT	Extracted quoted reply content (lines beginning with > or similar markers).
headings	TEXT	Extract all the headings present in the email content if available

4. Tasks

Task 1: Data Extraction Pipeline

Build a script or application that:

1. Recursively traverses the Enron maildir folder structure and discovers all email files.
2. Parses each raw email file to extract the mandatory and optional fields defined in Section 3.
3. Handles parsing edge cases gracefully: malformed headers, missing fields, encoding issues, multi-line header values, and nested forwarded messages.
4. Logs all parse failures with the file path and reason to a separate error log.
5. Outputs extraction statistics: total files found, successfully parsed, failed, and field-level completeness rates.

Task 2: Database Storage

Design and implement a local database to store the extracted data:

6. Use SQLite or PostgreSQL (local). Provide the schema as a .sql file or migration script.
7. The schema must properly normalize the data — for example, email addresses in to/cc/bcc should be stored in a related table, not as comma-separated strings.
8. Implement a unique constraint on message_id to prevent exact duplicates on insert.
9. Include indexes on date, from_address, and subject for efficient querying.
10. Provide at least 3 sample queries that demonstrate the database works correctly (e.g., count emails per sender, find all emails in a date range, find emails with CC recipients).

Task 3: Duplicate Detection & Flagging

Implement a duplicate detection system with the following logic:

Definition of a Duplicate: Two or more emails are considered duplicates if they share the same from_address, subject (after normalizing Re:/Fwd: prefixes), and body content (fuzzy match with at least 90% similarity). Among a set of duplicates, the one with the latest timestamp

is identified as the duplicate that needs confirmation, since it is likely a re-send or accidental repeat. The earliest email in the group is treated as the original record.

Requirements:

11. Scan the database after ingestion to identify all duplicate groups.
12. For each duplicate group, mark the latest email (by date) as the duplicate and retain the earliest email as the original. Flag the latest email in the database by setting is_duplicate = true and populating a duplicate_of column that references the original (earliest) message_id.
13. If a group has more than two emails, flag all except the earliest as duplicates. The latest duplicate is the primary target for notification (Task 4), but all flagged records should appear in the report.
14. Generate a summary report (duplicates_report.csv) listing: duplicate_message_id, original_message_id, subject, from_address, duplicate_date, original_date, similarity_score.
15. Log duplicate detection statistics: total groups found, total emails flagged, average group size.

Task 4: Send Notification Emails via MCP

This task requires you to configure and use a Gmail MCP server to actually send duplicate notification emails from your pipeline to an email address (Preferably your own gmail account). This tests your ability to integrate external services with AI-assisted tooling.

4.1 MCP Server Setup

Set up a Gmail MCP server that Claude Code (or your chosen AI tool) can use to send emails. You may use any of the following MCP servers:

Include your MCP server configuration in the project (with credentials redacted). Provide a mcp_config.json.example file showing the structure.

4.2 Notification Email Logic

For each duplicate group, send a notification email to the sender of the latest (duplicate) email informing them that their message has been flagged. The notification must follow this template:

```
To: <sender_of_latest_duplicate>
Subject: [Duplicate Notice] Re: <original_subject>
Date: <current_timestamp>
```

```
References: <message_id_of_latest_duplicate>

This is an automated notification from the Email Deduplication System.

Your email has been identified as a potential duplicate:

  Your Email (Flagged):
    Message-ID:  <message_id_of_latest_duplicate>
    Date Sent:   <date_of_latest_duplicate>
    Subject:     <subject>

  Original Email on Record:
    Message-ID:  <message_id_of_earliest_original>
    Date Sent:   <date_of_earliest_original>

  Similarity Score: <similarity_percentage>%

If this was NOT a duplicate and you intended to send this email,
please reply with CONFIRM to restore it to active status.

No action is required if this is indeed a duplicate.
```

4.3 MCP Integration Documentation

In your AI_USAGE.md file, include a dedicated section on MCP integration that covers:

16. Which MCP server you chose and why.
17. Step-by-step setup instructions (how to configure credentials, register the MCP server with Claude Code or your AI tool).
18. How you prompted the AI tool to use the MCP send_email tool — include example prompts.
19. Any issues encountered during MCP setup or sending, and how you resolved them.
20. A screenshot or log excerpt showing at least one successful email send in live mode.

5. Technical Requirements

Requirement	Details
Language	Python 3.10+
AI Tool	Claude Code (preferred), GitHub Copilot, Cursor, or any AI coding assistant. Document which tool you used.
Database	SQLite (preferred for portability) or PostgreSQL (local).

MCP Server	A Gmail-compatible MCP server for email sending. Use an existing open-source server or build a custom one. Must support the send_email tool.
Dependencies	Must include a requirements.txt or package.json. No proprietary/paid libraries.
Fuzzy Matching	Use a recognized library. Document your choice.
Error Handling	The pipeline must not crash on malformed input. All exceptions should be caught, logged, and skipped.
Reproducibility	A single command (e.g., make run or python main.py) should execute the full pipeline end to end. Email sending requires --send-live flag.

6. AI Tool Usage Documentation

This is a critical part of the evaluation. We want to understand how you collaborate with AI tools, not just see the final code. Include a file called AI_USAGE.md in your submission that documents:

21. Tool Used: Name and version of the AI coding tool.
22. Prompting Strategy: How did you break down the problem for the AI? Did you prompt task-by-task or provide the full spec at once? Include 3–5 example prompts you used and explain why you structured them that way.
23. Iterations & Debugging: Describe at least 2 cases where the AI-generated code did not work on the first attempt. What went wrong? How did you refine your prompts or manually fix the issue?
24. What You Wrote vs. What AI Wrote: Provide a rough percentage breakdown and identify specific sections you wrote manually vs. AI-generated.
25. Lessons Learned: What worked well with AI assistance? What was harder than expected?

7. Deliverables

Submit a Git repository containing:

Item	Description
Source Code	All scripts/modules for extraction, storage, duplicate detection, and MCP email sending.

README.md	Setup instructions (including MCP server setup), how to run, and architecture overview.
AI_USAGE.md	Detailed AI tool usage documentation as described in Section 7, including MCP integration section.
schema.sql	Database schema definition file (must include is_duplicate, duplicate_of, notification_sent, notification_date columns).
sample_queries.sql	At least 3 sample queries with expected output descriptions.
mcp_config.json.example	MCP server configuration template with placeholder credentials.
output/replies/	Generated draft reply .eml files for all flagged duplicates (dry-run output).
output/send_log.csv	Log of all emails sent in live mode: timestamp, recipient, subject, status, error.
duplicates_report.csv	Summary CSV of all detected duplicates with similarity scores.
error_log.txt	Log of all parse failures with file paths and reasons.
requirements.txt	Python dependencies (or package.json for Node.js)